CS 173 - Lecture 20

Analyzing Algorithms

ex. Recursive algorithm foo. What does it do? Time complexity?

15.5 Recursive versus Iterative Algorithms

Here is the code for a mysterious algorithm named Foo.

```
00  Foo(n: non-negative integer)
01    if n = 0 or n = 1
02      return n
03    else
04      a := 0
05      b := 1
06      for i := 2 to n
07        temp := b
08        b := b + a
09        a := temp
10      end for
11    return b
12    end if
```

(lines 01–05: $o(1)$; lines 06–10: $o(n)$; lines 07–09: $o(n)$ $o(1)$; lines 11–12: $o(1)$)

n	ret
0	0
1	1
2	1
3	2

n = 3:

a	b	i	temp
0	1		
		2	1
1	1		
		3	1
1	2		

n =

1) What it does

calculates n^{th} fibonacci number

2) Time complexity

$o(n)$

ex. Recursive algorithm merge

```
01 merge(L1,L2: sorted lists of real numbers)
02   if (L1 is empty)                          <- base cases
03     return L2                               o(1)
04   else if (L2 is empty)
05     return L1
06   else if (head(L1) <= head(L2))            o(1)
07     return cons(head(L1),merge(rest(L1),L2))   <- m-1, n
08   else
09     return cons(head(L2),merge(L1,rest(L2)))   <- m, n-1
```

1) recursive call on merge
2) return

Suppose L_1 has m elements, L_2 has n elements, first merge has inputs of size $(m-1, n)$, second merge has input size of $(m, n-1)$

Recursive relationship regarding time and input size:

Suppose $T(x)$ represents time needed when input size is x

$x = m+n$

$T(1) = c$ base case (base cases are $o(1)$ so we can say $T(1) = c$)

$T(x) = T(x-1) + d$ recursive case (d → $o(1)$ stuff)

1) come up with recursive definition
2) do unrolling to find closed form
3) find time complexity of closed form

Unrolling:

$T(x) = T(x-1) + d$

$= T(x-2) + 2d$

$o(1) \to c$
$o(n) \to cn$
$o(n^2) \to cn^2$
$o(n \log n) \to cn \log n$

$= T(x-3) + 3d$

$\ldots = T(x-k) + kd$

Base case:

$x - k = 1$, so $k = x - 1$

$T(x) = T(1) + kd = c + (x-1)d = dx + c - d = \boxed{O(x)}$

ex. mergesort recursive algorithm

```
01 mergesort(L = a_1, a_2, ..., a_n: list of real numbers)
02     if (n = 1) then return L                          base case O(1)
03 else
04     O(1) m = ⌊n/2⌋                                    need to find nth element
05     O(n) L_1 = (a_1, a_2, ..., a_m)                   first half
06     O(n) L_2 = (a_{m+1}, a_{m+2}, ..., a_n)           second half
07     return merge(mergesort(L_1), mergesort(L_2))      recursive
```

first recursive call: $T(n/2)$ (n/2) — second recursive call: $T(n/2)$ (n/2)

know this is $O(n)$ from previous problem

Define $T(n)$:

$T(1) = c$ base case $O(1)$

$T(n) = \underbrace{T(n/2)}_{\text{recursive call \#1}} + \underbrace{T(n/2)}_{\text{recursive call \#2}} + \underbrace{dn}_{\text{from merge}} + \underbrace{e}_{\text{constant time operations, can exclude bc we alr know } O(n) \gg O(1)}$

$= 2 \cdot T(n/2) + dn$

$= 2\left(2 \cdot T\left(\frac{n}{4}\right) + \frac{dn}{2}\right) + dn = 4T\left(\frac{n}{4}\right) + 2dn$

$= 4\left(2 \cdot T\left(\frac{n}{8}\right) + \frac{dn}{4}\right) + 2dn = 8T\left(\frac{n}{8}\right) + 3dn$

$\ldots = 2^k \cdot T\left(\frac{n}{2^k}\right) + kdn$

base case:

$\frac{n}{2^k} = 1$, $n = 2^k$, $k = \log_2 n$

$= 2^{\log_2 n} \cdot T(1) + \log_2 n \cdot dn = nc + dn \log n$ (base of log doesn't really matter cuz we're just looking for time complexity)

$= O(n \log n)$ bc $O(n) \ll O(n \log n)$

Discussion Problems – 15.2, 15.3, 15.4, 15.5

15.2 Mystery Code I

In line 05 the procedure maxthree is calling itself on a version of the input list with the kth element (a_k) removed. Assume it takes constant time to temporarily remove a_k from the list. (Doing this in constant time actually requires some extra details that we are hiding for clarity.)

```
00 maxthree(a_1, ..., a_n) : list of n positive integers, n ≥ 3)
01     if (n = 3) return a_1 + a_2 + a_3   o(1)
02     else
03         bestval = 0   o(1)
04         for k = 1 to n                          n
05 o(n)        newval = maxthree(a_1, a_2, ..., a_{k-1}, a_{k+1}, ... a_n)
06             if (newval > bestval) bestval = newval
07         end for
08         return bestval
```

(a) Describe (in English) what maxthree computes.

(b) Suppose that $T(n)$ is the running time of maxthree on an input array of length n. Give a recursive definition of $T(n)$.

(c) How many leaf nodes are there in the recursion tree for $T(n)$? Briefly explain.

(d) Does maxthree run in $O(2^n)$ time? Briefly explain why or why not.

a) Finds sum of largest three numbers

b) $T(3) = c$ $\quad T(n) = nT(n-1) + dn + e$

c) $\frac{n!}{3!}$

d) nah, $o(n!)$

15.3 Mystery Code II

The procedure crunch takes an array of integers $a_1, a_2, \ldots, a_n$ (where $n \geq 1$) and returns an integer. Assume that dividing the array in two (line 05) requires only constant time.

$[3, -4, 5]$ $n=3$
$a_1 \quad a_2 \quad a_3$
$m = 1$
$0 = c(3) + c(-4, 5)$

```
00 crunch(a_1, ..., a_n : array of integers)
01     if (n = 1 and a_1 ≥ 0) return 1   o(1)
02     else if (n = 1) return 0   o(1)
03     else
04         m = ⌊n/2⌋   o(n)
05         output = crunch(a_1, ..., a_m) + crunch(a_{m+1}, ..., a_n)
                          n/2                    n/2
06         return output   o(1)
```

(a) Give a succinct English description of what crunch computes.

(b) Suppose that $T(n)$ is the running time of crunch on an input array of length n. Write down a recurrence for $T(n)$, including a base case. For simplicity, you may assume that n is a power of 2.

(c) What is the big-Θ running time of crunch? Justify your answer either by unrolling the recurrence or by drawing a (well-labeled) recursion tree.

a) Returns number of non-negative integers in the array

b) $T(1) = c$ $\quad$ base case

$T(n) = T(n/2) + T(n/2) + d = 2T\left(\frac{n}{2}\right) + d$

c) $= 2\left(2T\left(\frac{n}{4}\right) + d\right) + d = 4T\left(\frac{n}{4}\right) + 3d$

$= 4\left(2T\left(\frac{n}{8}\right) + d\right) + 3d = 8T\left(\frac{n}{8}\right) + 7d$

$\ldots = 2^k T\left(\frac{n}{2^k}\right) + (2^k - 1)d$

for base case:

$\frac{n}{2^k} = 1 \quad n = 2^k \quad k = \log_2 n$

closed form:

$= 2^{\log_2(n)} T(1) + (2^{\log_2 n} - 1)d = cn + (n-1)d = \boxed{\Theta(n)}$

15.4 Mystery Code III

Consider an array of n *distinct* real numbers $a_1, a_2, \ldots, a_n$. We say that the array has a *peak* at position k if the following two conditions hold for every position j between 2 and n:

(1) If $j \leq k$, then $a_{j-1} < a_j$.

(2) If $j > k$, $a_{j-1} > a_j$.

Consider the following procedure to determine position of the peak of an array (assume that the array does indeed have a peak):

-1, 3, 6, 7, 0 n=5

```
00 procedure FindPeak(a1, a2, ..., an: array of real numbers)
01   if (n = 1)                        ×
02     return 1
03   if (a1 > a2)                      -1 > 3 ?  ×
04     return 1
05   else if (an > an-1)               0 > 7  ×
06     return n
07   k = floor((1+n)/2)                k = 3
08   if (ak-1 > ak)                    3 > 6  ×   n/2
09     return FindPeak(a1, ..., ak-1)
10   else if (ak < ak+1)               6 < 7 ✓   n/2
11     return FindPeak(ak+1, ..., an) + k
12   else                              FindPeak(7, 0) + 3
13     return k                        = 1+3 = 4
```

(base cases: lines 01–05)

(2 = 1 ? ×; 7 > 0 ✓ return 1)

$a_4 = 7$, which is peak

(a) Consider the array $-1, 3, 6, 7, 0$. Trace the execution of the above pseudocode and show that it correctly returns the position of the peak.

(b) At line 07, what is the smallest value that n might contain? Why?

(c) Let $T(n)$ be the worst-case running time of the above pseudocode when the array has size n. Write a recurrence for $T(n)$, including the necessary base case(s). Assume that splitting the array (lines 09 and 11) takes constant time.

(d) What is the big-Θ running time of FindPeak? Justify your answer by solving the recurrence via unrolling or use of a recursion tree.

b) 3, because can't be 1 from line 01 and can't be 2 from lines 04 and 06

c) $T(1) = c$

$T(2) = d$

$T(n) = T\left(\frac{n}{2}\right) + e$

d) $= T\left(\frac{n}{4}\right) + 2e$

$= T\left(\frac{n}{8}\right) + 3e$

$\ldots = T\left(\frac{n}{2^k}\right) + ke$

base case:

$1 = \frac{n}{2^k} \quad k = \log_2 n \quad$ or $\quad 2 = \frac{n}{2^k} \quad k = \log_2\left(\frac{n}{2}\right)$

closed form:

$= T(1) + c\log_2 n$ or $= T(2) + c\log_2(\frac{n}{2})$

$= c + c\log n$ or $= d + c\log(\frac{n}{2})$

$= \theta(\log n)$

15.5 Recursive versus Iterative Algorithms

Here is the code for a mysterious algorithm named Foo.

```
00  Foo(n: non-negative integer)
01     if n = 0 or n = 1
02        return n
03     else
04        a := 0
05        b := 1
06        for i := 2 to n
07           temp := b
08           b := b + a
09           a := temp
10        end for
11     return b
12     end if
```

(a) Give a brief English description of what the function Foo computes.

(b) What is the big-O running time of Foo? Justify your answer.

(c) A simple recursive version of Foo exists, which computes the value of Foo(n) using the values Foo(n-1) and Foo(n-2). Write the corresponding pseudocode for RecursiveFoo. Briefly explain how RecursiveFoo works.

(d) Briefly justify why Foo is more efficient than RecursiveFoo.

a) Calculates n^{th} fibonacci number

b) $O(n)$ bc for loop

c) Recursive Foo(n: non-negative integer)

```
    if n=0 or n=1
        return n
    else
        return RecursiveFoo(n-1) + RecursiveFoo(n-2)
```

d) Recursive Foo is some exponential time complexity, Foo is linear